

Algoritmos y Estructuras de Datos

Problem Set UT04 – Grafos Dirigidos.

Se debe subir un archivo zip con las siguientes características:

Nombre del archivo: `equipo-{numero de equipo}-{nombre de docente}.zip`

El mismo debe contener todo el código generado e imágenes o documentos en pdf en aquellos ejercicios que no se resuelven en código.

Grafos dirigidos

Ejercicio 1

Una compañía de aviación nacional necesita representar los vuelos que opera en diferentes aeropuertos y calcular cuáles son los viajes de menor costo que permiten unir esos aeropuertos. Los aeropuertos en los que opera son: Artigas, Canelones, Colonia, Durazno, Florida, Montevideo, Punta del Este y Rocha.

Los vuelos operados son (origen, destino, distancia):

- Artigas, Rocha, 400;
- Canelones, Artigas, 500;
- Canelones, Colonia, 200;
- Canelones, Durazno, 170;
- Canelones, Punta del Este, 90;
- Colonia, Montevideo, 180;
- Florida, Durazno, 60;
- Montevideo, Artigas, 700;
- Montevideo, Canelones, 30;
- Montevideo, Punta del Este, 130;
- Punta del Este, Rocha, 90;
- Rocha, Montevideo, 270

El analista a quien se le ha encargado la tarea ha decidido que la mejor forma de solucionar el problema es mediante el uso de grafos dirigidos.

Parte 1

1. Represente el escenario mediante:

- a) Matriz de adyacencias (ponderada por minutos; usar $0/\infty$ donde corresponda).

- b) Lista de adyacencias indicando, para cada vértice, sus aristas salientes con pesos.
2. Dibuje el grafo dirigido resultante, rotulando vértices y pesos de las aristas.

Parte 2

Aplicando paso a paso el método de Dijkstra, calcular el vuelo de menor costo (distancia) entre el aeropuerto de Montevideo y cada uno los otros aeropuertos. ¿Cuál es el trayecto recorrido? (salida– escalas – destino)

Parte 3

1. Descargar el proyecto de la unidad temática e incluirlo en el proyecto del curso. Descargar.
2. Analise las interfaces
 - IGraph
 - IDirectedIGraph
 - IDirectedGraphAlgorithms
 - IGraphAlgorithms
3. Evaluar y considerar API de colecciones de Java para implementar la interfaz IDirectedIGraph. Considerar el uso de distinta estructuras de datos y evaluar el consumo de memoria y orden de tiempo de ejecución.

Ejercicio 2

La aerolínea sudamericana

Vuelo seguro cuenta con varias líneas que conectan diversas ciudades. La Tabla 1 muestra las conexiones actualmente existentes.

Diversas agencias de viajes, que emiten boletos de esta compañía, desean tener en todo momento información sobre los itinerarios más económicos entre cualquier par de ciudades.

Por otro lado, la aerolínea desearía saber cuál sería la ciudad más conveniente para instalar los servicios de mantenimiento de sus aviones.

El Analista de Sistemas de la Empresa ha identificado el Grafo Dirigido como muy apropiado para la representación del problema y la resolución de diferentes requerimientos de información típicos en estas situaciones.

Origen \ Destino	Montevideo	Colonia	Buenos Aires	Punta del Este
Montevideo	x	-	-	150
Colonia	300	x	-	390
Buenos Aires	400	200	x	-
Punta del Este	-	-	410	x

Cuadro 1: Tabla 1

Parte 1

1. Lee cuidadosamente el Escenario, dibuja el Grafo dirigido correspondiente
2. Analiza el orden temporal del algoritmo de Floyd visto en la Cátedra (10 min).
¿Es posible reducirlo? ¿Cómo impacta su orden en operaciones reales bajo este contexto?

Parte 2

1. Usando Floyd para Grafo Dirigido, calcula las distancias mínimas (costos mínimos) entre todas las parejas de puertos (considerando combinaciones), previniendo la recuperación de caminos.
2. ¿En qué puerto sería más conveniente instalar el taller de mantenimiento?

Parte 3

En base a lo obtenido en el Ejercicio 2, desarrolla el algoritmo de recuperación de caminos mínimos (predecesores/ruta)

Ejercicio 3

La aerolínea ha expandido sus operaciones y ahora tiene los siguientes vuelos, de acuerdo a la Tabla 2.

Origen / destino	Asunción	Buenos Aires	Curitiba	Montevideo	Porto Alegre	Rio de Janeiro	San Pablo	Santos
Asunción	x	1600	800	-	700	-	-	-
Buenos Aires	1600	x	-	200	1250	-	-	-
Curitiba	-	-	x	-	-	1500	-	-
Montevideo	-	200	-	x	1000	-	980	1200
Porto Alegre	-	-	-	-	x	-	-	-
Rio de Janeiro	-	-	1500	-	-	x	1800	1900
San Pablo	1200	-	600	2000	980	1800	x	220
Santos	-	-	-	-	1200	-	220	x

Cuadro 2: Matriz de costos

Parte 1

Se provee un conjunto de clases e interfaces básicas para implementar un Grafo Dirigido. Dicha interface provee los métodos necesarios para agregar vértices y aristas.

1. En equipo se deberá:
 - Analizar las estructuras de datos diseñadas. ¿Cuáles pueden ser las razones para la utilización de las diferentes estructuras de la API de colecciones para representar el grafo, el conjunto de vértices, las listas de adyacencias?
 - Una vez decidida las estructuras de datos para representar el Grafo Dirigido, implementar agregar vértice y arista.
 - Dadas estas estructuras de datos, construir un programa Java que permita:
 - crear un grafo para representar las conexiones aéreas de la aerolínea «VUELE SEGURO»

- cargar este grafo utilizando los archivos provistos «aeropuertos.txt» y «conexiones.txt»
2. **Floyd:** Implementar el algoritmo bajo la interfaz `IDirectedGraphAlgorithms` y que permita calcular los caminos mínimos entre todos los pares de aeropuertos posibles y:
- contestar interactivamente preguntas del tipo ¿cuál es la menor distancia posible para ir desde una ciudad x a otra y?
 - contestar interactivamente preguntas del tipo «indique el itinerario de menor costo para ir de la ciudad x a la ciudad y»
 - contestar interactivamente preguntas del tipo «si la aerolínea decide instalar un nuevo centro de mantenimiento y logística para sus aviones, ¿cuál sería el aeropuerto más apropiado para ello?»

Parte 2

Implemente el algoritmo Warshall bajo la interfaz `IDirectedGraphAlgorithms`.

Ejercicio 4

A efectos de mejorar la atención al cliente, la aerolínea ha decidido desarrollar una aplicación que permita conocer todos los vuelos posibles para unir un origen con un destino.

Parte 1

Dada la matriz que indica las conexiones existentes entre ciudades, dibuja el grafo correspondiente y su representación por lista de adyacencias. Aplicando el algoritmo de búsqueda en profundidad, desarrolla el bosque abarcador resultante de recorrer este grafo, comenzando por la ciudad «Asunción». Indica claramente los diferentes tipos de arcos encontrados.

Origen / destino	Asunción	Buenos Aires	Curitiba	Montevideo	Porto Alegre	Rio de Janeiro
Asunción	x	1600	800	-	700	-
Buenos Aires	1600	x	-	200	1250	-
Curitiba	-	-	x	-	-	1500
Montevideo	-	200	-	x	1000	-
Porto Alegre	-	-	-	1000	x	-
Rio de Janeiro	-	-	1500	-	-	x

Cuadro 3: Matriz de costos

Parte 2

Dada las interfaces dadas para ser implementadas en Java, e desea poder utilizar esta estructura para realizar un recorrido sistemático en profundidad de un grafo así representado. Se deberá entonces implementar los métodos necesarios para realizar esta recorrida, comenzando por el primer vértice representado.

Implementar el método `recorridoEnProfundidad` en la interfaz `IGraphAlgorithms`.

Pista: Utilizar métodos auxiliares para pasar otras estructuras auxiliares que guarden datos. Por ejemplo: nodos visitados.

Ejercicio 5

A efectos de mejorar la atención al cliente, la aerolínea ha decidido desarrollar una aplicación que permita conocer todos los vuelos posibles para unir un origen con un destino.

Parte 1

- Describe en lenguaje natural el algoritmo que es necesario desarrollar para satisfacer ese requerimiento (listar o imprimir todos los itinerarios que se puedan dar, entre una cierta ciudad origen y una ciudad destino).
- Utilizando este algoritmo, y dado el grafo del Ejercicio 1 de TA4, encuentra todas las conexiones entre Montevideo y Porto Alegre (cada conexión con su costo total). Ejecutar paso a paso su algoritmo, en papel.

Parte 2

Implementar el método `obtenerTodosLosCaminos` el cual recibe:

- Un vértice origen
- Un vértice destino
- Un grafo donde sus aristas tiene un peso

y retorna una lista de caminos, donde para cada camino se conoce:

- los vértices que conforman el camino
- el costo para realizar dicho camino

Ejercicio 6

Parte 1

1. Desarrollar en lenguaje natural y pseudocódigo de alto nivel un algoritmo sencillo que permita determinar si un grafo contiene ciclos (15 minutos).
2. Dada la interfaz y las clases auxiliares, implementar el método `tieneCiclos`, que retorna `true` si el grafo tiene ciclos (20 minutos).
3. Dados los archivos «conexiones_2.txt» y «aeropuertos_2.txt», ejecutar el programa e indicar si el grafo resultante contiene ciclos o no. (5 minutos)

Parte 2 - Camino crítico

La ordenación topológica o sort topológico es ampliamente utilizado en planificación y gestión de proyectos o procesos que tienen una gran cantidad de tareas. Las tareas suelen tener recursos asignados (tiempo, horas hombre, materia prima necesaria, etc.). Las tareas pueden verse como un cambio de estado de un cierto elemento. Existen naturalmente precedencias entre las tareas (ejemplo: antes de pintar la pared es necesario revocarla, y antes construirla, etc etc).

De todos los posibles «caminos» que representan las diferentes secuencias de tareas, existe al menos uno que, si cualquiera de las tareas que lo constituyen, se atrasa,

entonces todo el proyecto se atrasará. Este es el llamado «camino crítico» del proyecto (una herramienta muy utilizada en planificación es el PERT, «diagrama de camino crítico»).

Las demás secuencias (que no son camino crítico), podrán tener ciertas demoras en algunas de sus tareas, hasta alcanzar la magnitud del camino crítico. Por ejemplo, si una secuencia de tareas que es camino crítico tiene un costo total de 23 días (suma de los costos de las tareas o arcos), y otra secuencia tiene un costo o duración total de 15 días, esto significará que existe una «holgura» de 8 días, es decir, algunas de las tareas que componen esta secuencia podrán empezar y terminar en diferentes fechas, siempre que no excedan la final. Esto nos permite, al planificar los trabajos, buscar formas de distribuir los recursos existentes de manera más homogénea, evitar picos de necesidad de recursos o espacios ociosos.

Se desea desarrollar entonces un algoritmo que, dado un grafo (que debería ser acíclico) permita:

- calcular el costo total del camino crítico e indicar cómo está compuesto este camino, y
- listar todas las demás secuencias de tareas posibles, sus costos totales y la holgura existente
- deberán establecerse las estructuras apropiadas con las modificaciones que sean necesarias

Parte 3

Los grafos dirigidos acíclicos son modelos adecuados para representar órdenes parciales. Por ejemplo, el sistema de preinscripciones de las asignaturas de una carrera universitaria o un orden necesario en el que se deben ejecutar las tareas de un proyecto. En un proyecto modelado con un grafo dirigido en el que los vértices son las tareas y las aristas dirigidas representan las precedencias (existe un vértice de Fin del proyecto), se puede obtener una lista de vértices cuyo orden representa un orden posible de ejecución de las tareas del proyecto (contiene todas las tareas del proyecto).

Tarea Desarrollar el algoritmo **calcularClasificacionTopologica** definido en la interfaz **IDirectedGraphAlgorithms**.

Ejercicio 7

Parte 1

Dada la siguiente matriz de adyacencias:

1. Dibuja el grafo dirigido
2. Halla los caminos de menor costo, aplicando el algoritmo de Dijkstra paso a paso, desde el vértice B hasta todos los otros vértices
3. Indica cuáles son los caminos de menor costo utilizando un vector de predecesores
4. Escribe un algoritmo para, dado el vector de predecesores, imprimir el camino entre el vértice origen y un vértice destino indicado como parámetro.

	A	B	C	D	E
A	-	1	2	7	-
B	7	-	1	2	-
C	-	-	-	3	-
D	6	-	4	-	4
E	-	2	-	8	-

Parte 2

Dado el grafo de la figura, y utilizando el algoritmo de FLOYD:

- Dibuja la representación mediante lista de adyacencias.
- Encuentra los caminos de menor costo entre los vértices del grafo de la figura.
- Utilizando una matriz de predecesores, muestra cómo recuperar el camino entre un par de vértices dados.
- ¿Cuál es el camino de menor costo entre los vértices A y E? (utiliza el algoritmo, ejecutando paso a paso, para resolver).
- ¿Cuáles son las excentricidades de los vértices del grafo?
- ¿Qué vértice es el centro del grafo?

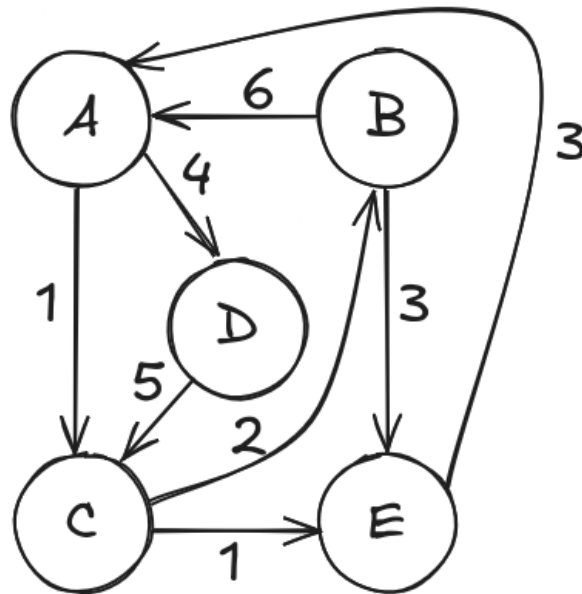


Figura 1: Grafo Dirigido

Grafos no dirigidos

Ejercicio 8

Una compañía de reforestación sembrará árboles en seis zonas en la misma área. Para esto debe desarrollar un sistema de caminos de tierra para tener acceso a cualquier zona desde cualquier otra. La distancia (en kilómetros) entre cada par de zonas está dada en la siguiente tabla:

	Zona 1	Zona 2	Zona 3	Zona 4	Zona 5	Zona 6
Zona 1	x	2	5	1	7	2
Zona 2	2	x	1	3	5	6
Zona 3	5	1	x	2	3	5
Zona 4	1	3	2	x	7	1
Zona 5	7	5	3	7	x	4
Zona 6	2	6	5	1	4	x

Se desea determinar la menor y más barata (cuanto más corto, menos costoso) cantidad de caminos que permita conectar todas las zonas entre sí.

Parte 1

1. Describe en lenguaje natural qué estructuras, conceptos y algoritmos de la Unidad Temática pueden aplicarse para resolver este problema práctico.
2. Representa la estructura elegida mediante Lista de adyacencias.
3. Representa gráficamente la red de zonas y distancias.
4. Determina la mejor opción para la construcción de los caminos de acuerdo con el escenario, utilizando el algoritmo de **Prim** mostrando cada paso realizado.

Parte 2

1. Partiendo de la base del pseudocódigo abstracto del algoritmo de PRIM que vimos en clase, desarrollar el algoritmo en pseudocódigo detallado (¡¡listo para ser implementado en JAVA!!). Puedes utilizar como estructuras auxiliares para el desarrollo del algoritmo solamente las vistas hasta el momento en el curso (listas, árboles, grafos, colecciones de JAVA, etc.). Tener en cuenta los criterios estándar para la escritura de pseudocódigo publicados.
2. Prueba paso a paso la correcta ejecución del algoritmo, utilizando un escenario reducido (por ejemplo, sólo 4 zonas)
3. Desarrolla en pseudocódigo y lenguaje natural la especificación de los casos de prueba que estimes necesarios para una segura implementación del algoritmo.
4. Preparar POSTER con algoritmo y casos de prueba

NOTA : Este algoritmo y casos de pruebas, en pseudocódigo, serán necesarios para la realización del Trabajo de Aplicación 2 en la próxima clase.

Ejercicio 9

Una compañía de reforestación sembrará árboles en seis zonas en la misma área. Para esto debe desarrollar un sistema de caminos de tierra para tener acceso a cualquier zona desde cualquier otra. La distancia (en kilómetros) entre cada par de zonas está dada en la siguiente tabla:

	Zona 1	Zona 2	Zona 3	Zona 4	Zona 5	Zona 6
Zona 1	x	3	5	1	7	2
Zona 2	3	x	1	3	5	6
Zona 3	5	1	x	2	3	5
Zona 4	1	3	2	x	7	1
Zona 5	7	5	3	7	x	4
Zona 6	2	6	5	1	4	x

Se desea determinar la menor y más barata (cuanto más corto, menos costoso) cantidad de caminos que permita conectar todas las zonas entre sí.

Parte 1

1. Represente gráficamente la red de zonas y distancias.
2. Determine la mejor opción para la construcción de los caminos de acuerdo al escenario, utilizando el algoritmo de KRUSKAL, mostrando la ejecución paso a paso.

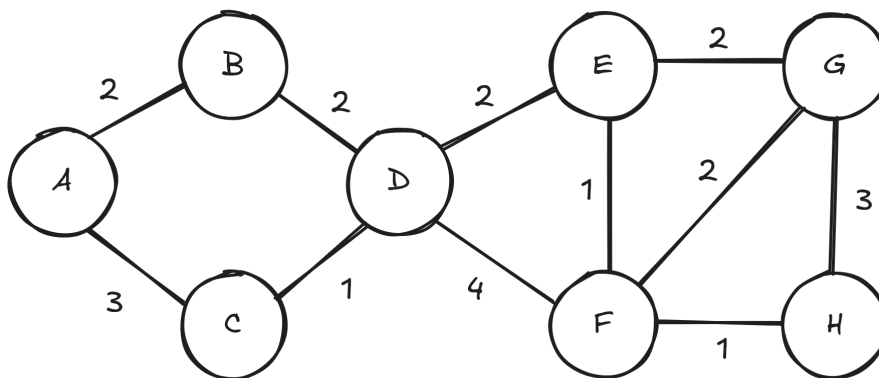
Ejercicio 10

Una empresa productora de lácteos desea realizar un proceso para asegurarse de que recoge la leche de todos los productores que se encuentran en su red. Para ello debe asegurarse de que los visita sistemáticamente, y una vez a cada uno.

El analista de la empresa ha considerado que el problema puede representarse mediante un grafo no dirigido, y que éste se puede recorrer tanto utilizando método de búsqueda en profundidad como en amplitud.

Parte 1

Dado el siguiente grafo no dirigido de ejemplo, realiza el recorrido del mismo mediante búsqueda en amplitud, indicando el árbol abarcador resultante, a partir del vértice con etiqueta «E».



Parte 2

Desarrolla un algoritmo, en pseudocódigo, para recorrer la estructura mediante búsqueda en amplitud.

NOTA Observa con cuidado las precondiciones.

Parte 3

Utilizando las estructuras de datos ya implementadas, desarrolla la «búsqueda en amplitud».

Ejercicio 11

Una empresa de telecomunicaciones ha establecido varios canales de intercambio de datos entre una cantidad de ciudades en África. Los centros de cómputos de interconexión se encuentran en estas ciudades:

- Nairobi (Kenya)
- Cairo (Egipto)
- Monrovia (Liberia)
- Garoua (Camerún)
- Mekele (Etiopía)
- Praia (Cabo Verde)

El analista a cargo del desarrollo de los sistemas críticos ha entendido que una buena representación de la red puede desarrollarse utilizando la estructura de Grafos No Dirigidos.

Uno de los problemas más acuciantes para la empresa es poder determinar eficientemente cuáles nodos de la red, en caso de falla (por ejemplo, por falta en la alimentación eléctrica), determinan la aislación de partes de la red.

La siguiente tabla indica las conexiones existentes entre las ciudades

- NAIROBI – CAIRO
- NAIROBI – MONROVIA
- NAIROBI – GAROUA
- MONROVIA – GAROUA
- MONROVIA – MEKELE
- GAROUA – MEKELE
- MEKELE – PRAIA

Parte 1

1. Dibuje el grafo representativo de esta red.
2. De acuerdo al método para hallar los “puntos de articulación” de un grafo, encuentre qué ciudades, en caso de faltar la energía eléctrica o los servicios de comunicaciones, podrían afectar la conectividad de la red de la empresa.

Parte 2

1. Desarrolla en pseudocódigo un algoritmo detallado que, dado una estructura de Grafo No Dirigido, encuentre los puntos de articulación.
2. Describir el algoritmo, indicando precondiciones y lenguaje natural .
3. El algoritmo **debe utilizar los métodos definidos por defecto**.

Parte 3

Implementar el algoritmo para hallar los puntos de articulación de un grafo no dirigido conexo. La firma del método es el siguiente:

```
// V tipo del nodo, E tipo de la arista
<V, E> List<V> puntosDeArticulacion(IGraph<V, E> grafo);
```

Ejercicio 12

El objeto del juego Kevin Bacon consiste en vincular a un actor con Kevin Bacon a través de películas en las que hayan trabajado juntos. El número mínimo de vínculos es el número de Bacon de un actor. Por ejemplo, Tom Hanks tiene un número de Bacon de 1, pues trabajó con Kevin Bacon en Apolo 13. Sally Field tiene un número de Bacon 2 porque trabajó en Forest Gump con Tom Hanks, que a su vez trabajó en Apolo 13 con Kevin Bacon. Casi todos los actores más conocidos tienen un número de Bacon de 1 o 2.

Supongamos que hemos decidido representar estas relaciones mediante un grafo no dirigido, y contamos con una lista de actores y otra de relaciones entre los mismos (la relación entre dos actores directa sería una película y, para simplificar, añadiremos la restricción que dos actores sólo pueden estar directamente relacionados en una película).

Parte 1

Dado el TDA de grafo no dirigido con las características y operaciones habituales, donde los vértices representan actores (con tipo String) y las aristas indican la película en que trabajan dos actores (con tipo String e indicando el nombre de la película), se deberá:

1. Implementar un método **numBacon** que, dado un cierto actor (tipo String) encuentre su número de Bacon (existe un actor representado en el grafo con nombre «Kevin_Bacon»).
2. Implementar un programa principal que:
 - a) Cargue el grafo con los vértices contenidos en el archivo «actores.csv».
 - b) Cargue las aristas contenidas en el archivo «en_películas.csv».
 - c) Invoque la operación “numBacon” del grafo, con los parámetros indicados más abajo, emitiendo por consola el resultado.

3. Ejecuta el programa y anota en esta hoja el resultado de las invocaciones. Registra este mismo resultado en el archivo «salida.txt» que has de adjuntar al código completo.
4. Desarrolla los casos de prueba necesarios para verificar la corrección de los métodos implementados.

Número de Bacon para los siguientes actores:

- John_Goodman
- Tom_Cruise
- Jason_Statham
- Lukas_Haas
- Djimon_Hounsou